Protein-Ligand Binding Site Prediction

Progress Summary

Project Collaboration Report

January 12, 2026

Abstract

This document summarizes the progress made in developing machine learning models for protein-ligand binding site prediction using the PDBbind database. The project explored multiple approaches, including Random Forest, Fully Connected Neural Networks (FCNN), Transformer classifiers, and Context-Aware Transformers, each with varying levels of success in predicting amino acid-level binding sites.

Contents

1 Project Overview **3**
- 1.1 Objective . . . 3
- 1.2 Data Source . . . 3
- 1.3 Problem Formulation . . . 3

2 Feature Engineering **3**
- 2.1 Baseline Feature Set (46 Dimensions) . . . 3
- 2.2 Enhanced Feature Set (466 Dimensions) . . . 3

3 Model Development **4**
- 3.1 Random Forest Classifier . . . 4
  - 3.1.1 Approach . . . 4
  - 3.1.2 Results . . . 4
  - 3.1.3 Key Findings . . . 4
- 3.2 Fully Connected Neural Network (FCNN) . . . 4
  - 3.2.1 Approach . . . 4
  - 3.2.2 Training Configuration . . . 4
- 3.3 Transformer Classifier . . . 5
  - 3.3.1 Architecture . . . 5
  - 3.3.2 Variants . . . 5
- 3.4 Context-Aware Transformer . . . 5
  - 3.4.1 Motivation . . . 5
  - 3.4.2 Architecture . . . 5
  - 3.4.3 Key Components . . . 5
  - 3.4.4 Alternative: CLS Token Architecture . . . 5

4 Regression Experiments **6**
- 4.1 Log-Transform Distance Regression . . . 6
  - 4.1.1 Approach . . . 6
  - 4.1.2 Threshold-Based Classification . . . 6
  - 4.1.3 Findings . . . 6

5 Training Infrastructure **6**
5.1 Loss Functions . . . 6
5.2 Data Pipeline . . . 6
5.3 Hardware . . . 6

6 Saved Models **6**

7 Code Organization **7**
7.1 Source Code Structure . . . 7
7.2 Training Scripts . . . 7
7.3 Evaluation Scripts . . . 7

8 Conclusions and Future Directions **8**
8.1 What Worked . . . 8
8.2 Challenges . . . 8
8.3 Recommended Next Steps . . . 8

9 Timeline **8**

1 Project Overview

1.1 Objective

The main goal of this project is to develop computational methods for predicting protein-ligand binding sites at the residue (amino acid) level. Given a protein sequence and ligand properties, the models predict which amino acids are likely to be involved in binding interactions.

1.2 Data Source

The project uses the **PDBbind database**, a comprehensive collection of protein-ligand complexes with experimentally determined structures and binding affinities.

1.3 Problem Formulation

- **Input**: Amino acid features + global protein features + ligand properties
- **Output**: Binary classification (binding / non-binding) per residue
- **Challenge**: Highly imbalanced data (binding sites are rare)

2 Feature Engineering

2.1 Baseline Feature Set (46 Dimensions)

The initial feature extraction pipeline generates 46-dimensional feature vectors:

Index Range	Feature Group	Description	Dims
0–19	Residue Identity	One-Hot Encoding (20 amino acids)	20
20–21	Residue PhysChem	Charge and Hydrophobicity	2
22–41	Global Protein	Amino Acid Composition (AAC)	20
42–45	Ligand Features	LogP, MW, H-Donors, H-Acceptors	4

Table 1: Baseline 46-dimensional feature vector composition

2.2 Enhanced Feature Set (466 Dimensions)

The enhanced pipeline adds sophisticated protein sequence descriptors:

Index Range	Feature Group	Description	Dims	Source
0–19	Residue Identity	One-Hot Encoding	20	Custom
20–21	Residue PhysChem	Charge, Hydrophobicity	2	Custom
22–41	AAC	Amino Acid Composition	20	pypropel
42–441	DAC	Dipeptide Composition	400	pypropel
442–461	AVEANF	Amphiphilic Pseudo-AA	20	pypropel
462–465	Ligand Features	LogP, MW, H-Donors, H-Acceptors	4	RDKit

Table 2: Enhanced 466-dimensional feature vector composition

3 Model Development

3.1 Random Forest Classifier

3.1.1 Approach

The first model implemented was a Random Forest classifier, chosen for its:

- Robustness to feature scaling
- Built-in feature importance
- Ability to handle imbalanced data

3.1.2 Results

Configuration	Data Size	ROC-AUC	Accuracy	Recall	Precision
Baseline (46 dims)	5 Batches (200k)	**0.8013**	0.8330	0.54	0.20
Enhanced (466 dims)	5 Batches (227k)	0.7248	**0.8687**	0.36	0.15
Enhanced (466 dims)	Full (569k)	**0.7961**	0.8387	**0.55**	0.19

Table 3: Random Forest model comparison (December 27, 2025)

3.1.3 Key Findings

1. **Baseline Effectiveness**: The simple 46-dim feature set is robust (AUC ∼0.80) even with limited data.
2. **Curse of Dimensionality**: Enhanced features underperformed when data was limited (AUC 0.72 vs 0.80).
3. **Data Scale Matters**: Full dataset allowed enhanced features to recover performance.
4. **Performance Ceiling**: More features did not strictly outperform baseline, suggesting need for more complex models.

3.2 Fully Connected Neural Network (FCNN)

3.2.1 Approach

A simple multi-layer perceptron was implemented to establish a neural network baseline, using the enhanced 466-dimensional features.

3.2.2 Training Configuration

- Focal Loss to handle class imbalance
- Adam optimizer with learning rate scheduling
- Early stopping based on validation AUC

3.3 Transformer Classifier

3.3.1 Architecture

The Transformer-based classifier treats the 466-dimensional feature vector as a sequence of feature patches:

1. **Input Projection**: Features divided into patches (∼20 features each)
2. **Positional Encoding**: Sinusoidal encoding for patch positions
3. **CLS Token**: Learnable classification token prepended
4. **Transformer Encoder**: Self-attention layers learn feature interactions
5. **Classification Head**: MLP with sigmoid output

3.3.2 Variants

- **BindingTransformer**: Full architecture with patch-based input
- **BindingTransformerLite**: Lighter version with direct projection

3.4 Context-Aware Transformer

3.4.1 Motivation

To leverage sequence context for binding prediction, treating each prediction as dependent on neighboring amino acids.

3.4.2 Architecture

1. **Input**: Window of $2k + 1$ amino acids (context size k)
2. **Learnable Positional Encoding**: For short sequences
3. **Self-Attention**: Learns inter-residue relationships
4. **Center Token Extraction**: Prediction for center residue
5. **Classification Head**: Binary binding probability

3.4.3 Key Components

```
class ContextTransformer(nn.Module):
    # context_size: Residues on each side of center
    # d_model: Transformer dimension (256)
    # nhead: Attention heads (8)
    # num_layers: Encoder layers (4)
```

3.4.4 Alternative: CLS Token Architecture

Also implemented `ContextTransformerWithCLS` that uses a learnable CLS token for classification instead of the center token.

4 Regression Experiments

4.1 Log-Transform Distance Regression

An alternative approach was explored: predicting the distance to the nearest ligand atom as a regression task.

4.1.1 Approach

- Target: Log-transformed distance to nearest ligand atom
- Model: Transformer-based regression architecture
- Loss: Mean Squared Error (MSE)

4.1.2 Threshold-Based Classification

Post-hoc classification by applying distance thresholds to regression predictions.

4.1.3 Findings

- Model struggled to predict very short distances (actual binding sites)
- MSE loss dominated by large distances
- Direct classification remains more effective for this task

5 Training Infrastructure

5.1 Loss Functions

- **Focal Loss**: Addresses class imbalance by down-weighting easy examples
- **Weighted BCE**: Class-weighted binary cross-entropy
- **MSE**: For regression experiments

5.2 Data Pipeline

- Batch processing of PDB structures
- Protein-level train/val/test split (prevents leakage)
- Context window generation for sequence models
- On-the-fly data loading for memory efficiency

5.3 Hardware

- Training on Apple Silicon (MPS) when available
- CUDA support for GPU clusters
- CPU fallback

6 Saved Models

The following trained models are available:

Model File	Description
rf_model.joblib	Random Forest classifier
rf_model_enhanced.joblib	RF with enhanced features
rf_model_enhanced_full.joblib	RF on full dataset
best_fcnn_focal.pth	FCNN with Focal Loss
best_transformer_focal.pth	Transformer classifier
best_transformer_lite_focal.pth	Lite Transformer
best_context_transformer_focal.pth	Context-Aware Transformer
best_regression_transformer_mse.pth	Distance regression model
best_regression_transformer_mse_log.pth	Log-distance regression

Table 4: Saved model checkpoints

7 Code Organization

7.1 Source Code Structure

```
src/
|-- dataset.py              # Base dataset loader
|-- dataset_context.py      # Context window dataset
|-- dataset_regression.py   # Regression target dataset
|-- features.py             # Feature extraction
|-- loss.py                 # Custom loss functions (Focal)
|-- model.py                # Base model definitions
|-- model_context_transformer.py   # Context-aware models
|-- model_regression.py     # Regression models
|-- model_rf.py             # Random Forest wrapper
|-- model_transformer.py    # Transformer classifiers
|-- pipeline.py             # Training pipeline
|-- pipeline_regression.py  # Regression training
+-- utils.py                # Utility functions
```

7.2 Training Scripts

- `train_fcnn.py` – FCNN training
- `train_transformer.py` – Transformer training
- `train_context_transformer.py` – Context-aware training
- `train_regression.py` – Distance regression training

7.3 Evaluation Scripts

- `evaluate_fcnn.py`
- `evaluate_transformer.py`
- `evaluate_context_transformer.py`
- `evaluate_regression.py`

8 Conclusions and Future Directions

8.1 What Worked

1. **Baseline features** (46 dims) provide strong performance with simple models
2. **Focal Loss** effectively handles class imbalance
3. **Protein-level data splitting** prevents data leakage
4. **Context-aware modeling** captures sequence dependencies

8.2 Challenges

1. **High-dimensional features** require more data to avoid overfitting
2. **Class imbalance** remains a fundamental challenge
3. **Precision-recall trade-off** difficult to optimize

8.3 Recommended Next Steps

1. **Feature Selection**: Apply feature selection on the 466-dim set to remove noise
2. **Gradient Boosting**: Explore XGBoost/LightGBM for high-dimensional features
3. **Structure Features**: Incorporate 3D structural information
4. **Pre-trained Embeddings**: Use protein language models (ESM, ProtTrans)
5. **Multi-task Learning**: Joint prediction of binding sites and affinity

9 Timeline

Date	Milestone
Dec 27, 2025	Feature extraction pipeline completed
Dec 27, 2025	Random Forest PoC with baseline & enhanced features
Dec 27, 2025	FCNN implementation and training
Dec 27, 2025	Transformer classifier development
Jan 5–6, 2026	Log-transform regression experiments
Jan 12, 2026	Context-Aware Transformer implementation and evaluation

---

Document generated on January 12, 2026